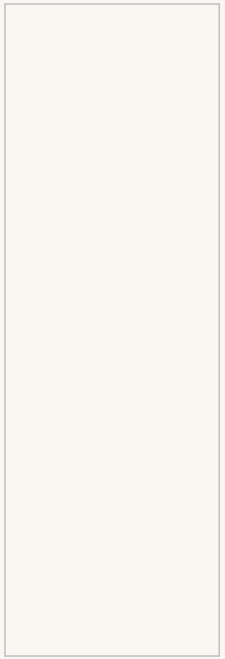
Agentic Power BI Development

From PBIP to Pair Programmer

Nicky van Vroenhoven · Daan Humblé

dataMinds Connect · Mechelen · Wednesday, 14 October 2026



This chapter shows how Copilot can help with a small change in a Power BI project when the model and report definitions are available as text. The change is reviewed and verified in Power BI Desktop.

Start with a real Power BI project

OLD KETTLE BREWING

STARTING POINT

Existing PBIP project

A real model, already in use.

→

TODAY'S SMALL CHANGE

One Revenue measure

Copilot Chat writes TMDL. Review the diff. Verify in Desktop.

NOT THIS SESSION Copilot in Power BI · Fabric Apps · Data Agents · Ontology

Ch01 demonstrates one small change in an existing Old Kettle Brewing PBIP project: Copilot Chat and inline assistance help write a Revenue measure in TMDL; a human reviews the text diff; then the project is reopened in Power BI Desktop for verification.

This chapter does not cover Copilot in Power BI, Fabric Apps, Data Agents or Ontology.

Meet your guides



OPENING THE SESSION

Nicky van Vroenhoven

Unit Lead Fabric & Power BI

Powerdobs B.V.

LinkedIn /in/nickyvanvroenhoven

Recognition Data Platform MVP



TAKING YOU FURTHER

Daan Humblé

Cloud Solution Architect · Data & AI

Microsoft

LinkedIn /in/daanhumble

GitHub /HumbleDaan

Nicky van Vroenhoven is Unit Lead Fabric & Power BI at Powerdobs B.V. and a Data Platform MVP. Daan Humblé is a Cloud Solution Architect for Data & AI at Microsoft. Their LinkedIn profiles are listed; Daan's GitHub profile is also included.

DEMO LIVE

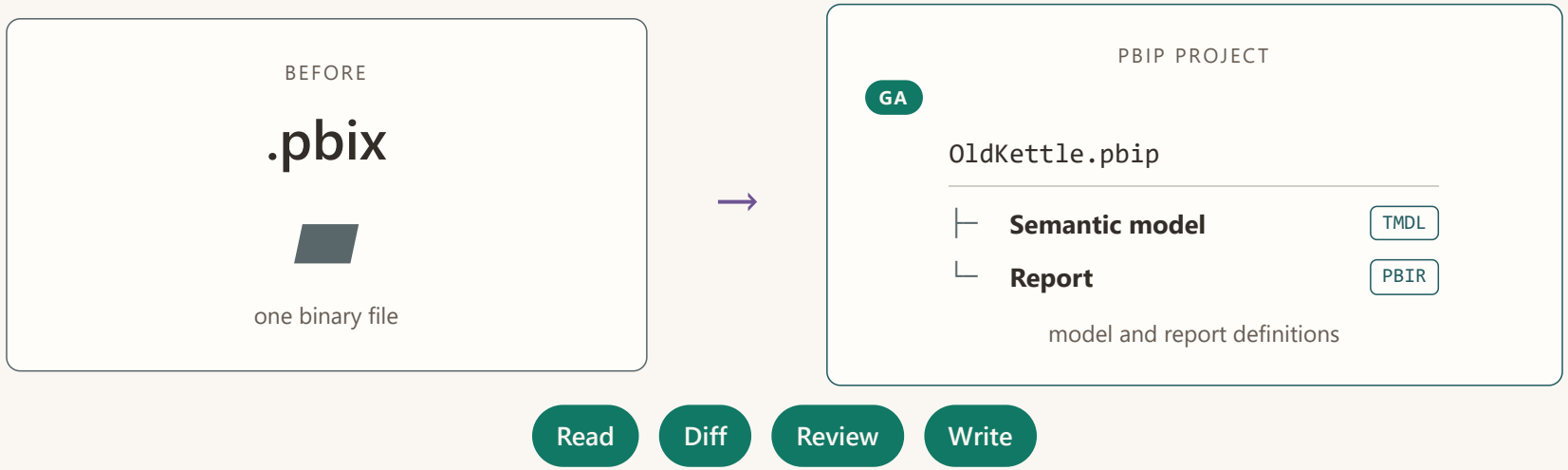
Old Kettle Brewing

Edit in TMDL → review the diff → check Desktop

The demonstration uses Copilot Chat and inline assistance to create a Revenue measure and another measure for the Quantity column in a local TMDL file, followed by a human-authored description and Total COGS measure. A Git diff makes the changes inspectable before the updated model is checked in Power BI Desktop.

The recorded path is a fallback, not evidence that the live run succeeded. Keep the measure in its local table; Ch03 later

The model and report became text



PBIP represents a Power BI project as project files rather than only as a single PBIX binary. Its model and report definitions use text-based formats, including TMDL and PBIR. Those definitions can be inspected and compared as text in source control and edited by text-capable tools.

Microsoft documents Power BI Desktop projects (PBIP) and PBIR as generally available. Detecting and reloading external PBIP

The model is readable to a text tool

OldKettle.SemanticModel

- model.tmdl
- tables/
  - Sales.tmdl
- cultures/
- perspectives/

```
01  table Sales
02   column Revenue
03   measure Total COGS
04   measure Total Quantity
```

declarative · modular · versionable

TMDL is the text-based model definition used in a PBIP project. Model objects such as tables, columns and measures can be represented in versionable files. A text editor and source-control tools can therefore inspect the model changes without treating the whole project as an opaque binary.

The report is text too

```
OldKettle.Report
  report.json
  pages/
    Overview/
      visuals/
  bookmarks/
```

A REPORT DEFINITION

Visual	title · fields · sort
Page	layout · visuals

PBIR is the text-based report-definition format used by PBIP projects. Report definitions can describe pages and visuals. In this chapter, any optional edit is checked by reopening Desktop; automatic Desktop reload belongs to the later Desktop Bridge chapter.

The workspace sits beside the project

VS CODE WORKSPACE

Fabric VS Code extension

```
▸ Power BI workspace
  OldKettle.pbip
  • model · report · project files
```

One place to work with the project files

The Fabric VS Code extension is the editor-side entry point named in this chapter’s source. The essential point is simply that the project files are available alongside the code and tools used to work with them. Verify the product name and current capabilities before presenting them.

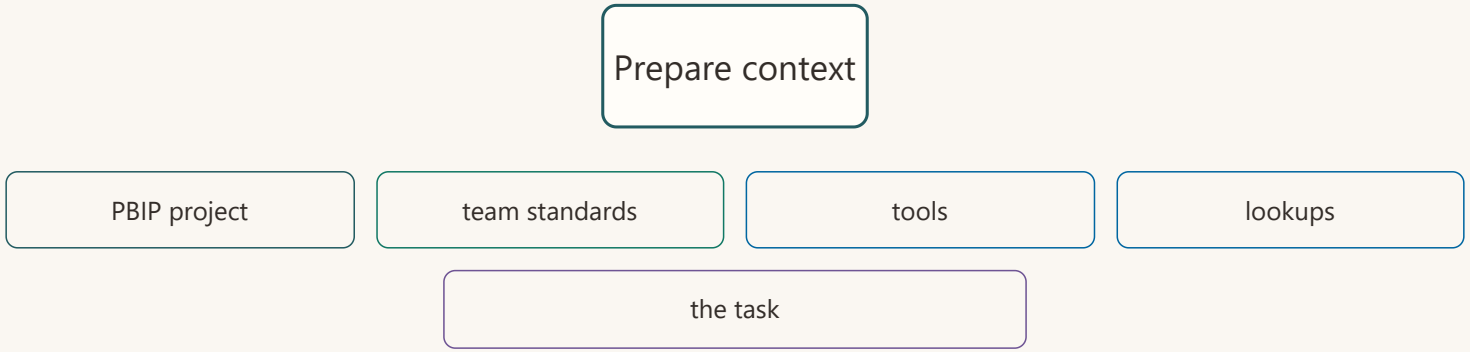
Four rungs. One increasing investment.

04	**Delegate**	Run work you did not watch
03	**Equip**	Look things up instead of guessing
02	**Teach**	Work the way your team works
01	**Show**	Complete what you already do

I'm on the ground floor. Daan takes you up from here.

The talk follows an escalating ladder: Show demonstrates completion; Teach supplies team standards; Equip connects the assistant to tools and lookups; Delegate hands work to a coordinated set of agents. Each chapter raises both the setup investment and the potential return.

Context shapes the answer



A better answer already starts before the prompt.

Mise en place means preparing what is needed before work begins. In an AI-assisted project, that preparation includes the project files, team standards, tools and relevant lookups. The task is given after that context is ready. Ch02 focuses on teaching the assistant the team's way of working.

It can write the measure. Does it fit your house?

WORKS

Valid measure

Does it calculate?

≠

FITS OUR HOUSE

Team convention

Format · description · naming

Next: teach it how we work.

A generated measure can be syntactically valid without following a team's naming, format or description conventions. Review the real output against standards the team has actually agreed. If no mismatch is observed, do not claim that one occurred; an unspecified convention is a missing instruction, not proof of a model error.

This is the transition to Ch02: teach the assistant how the team works.

What I teach mine

ME · MONDAY
Revenue is *after* discount, not list price.

ME · WEDNESDAY
No — *our* Revenue. After discount.

ME · FRIDAY
Again: Revenue is after discount.

`.github/skills/…/SKILL.md`

Revenue = what customers paid, after discounts.

Said it twice? It's a skill.

A skill starts where repetition starts. When you find yourself explaining the same thing to an AI assistant more than once — what "Revenue" means in your organisation, how your reports look, which numbers matter — write it down once, as a skill, in the repository.

A skill is a folder with a `SKILL.md` file: plain Markdown that the agent loads when the task matches its description.

Two skills in this repo

TECHNICAL · HOW

Install it

`report-authoring`

How to change a report: PBIR, validate, the gotchas.

From skills-for-fabric — trimmed for this demo.

FUNCTIONAL · WHAT

Write it

`old-brewery-house-style`

Theme | Brand rail | Our words | Never do

Ours. Nobody else can write it.

Technical skills teach an agent how to do a job correctly — for Power BI reports, the PBIR file format, validation and known pitfalls. They are largely shared: `report-authoring` comes from Microsoft's skills-for-fabric, trimmed for this demo.

Functional skills teach it how your team works: house style, page structure, what business terms mean, what you never do. Only you can write them. Here: `old-brewery-house-style`.

A skill is a folder — it ships our theme and bottle.

OLD KETTLE SALES
- .github
  - copilot-instructions.md
  - skills
    - report-authoring
    - old-brewery-house-style
      - SKILL.md
      - assets
        - old-brewery-theme.json
        - old-brewery-bottle.svg
- Old Kettle Sales.Report
- Old Kettle Sales.SemanticModel

```
---
name: old-brewery-house-style
description: >-
  WHAT a Power BI report looks like at the Old
  Brewery: our theme, page grid, brand rail with
  the bottle, chart rules and what our business
  terms mean. … Use whenever you restyle, revamp
  or fix the layout of a report page.
---
## 0. Restyle an existing report — the fast path
## 1. The look        ## 2a. The brand rail
## 7. What our words mean
## 8. What we never do
```

The description is the trigger.

A skill's front-matter `description` decides when the agent loads it: the agent reads every description, and loads the full body only when the request matches. The body holds the rules; an `assets/` folder can ship files the agent uses — here the house theme and the brand bottle, which no model could know or invent.

VS Code — Agent Skills

DEMO LIVE recording as fallback

With our style

> The layout of the Sales Overview page is a mess — fix it.

Theme · 1920 × 1080 · Brand rail + bottle · The grid · Donut → bar, no legends · KPI + vs previous year · Title = the answer · Validate

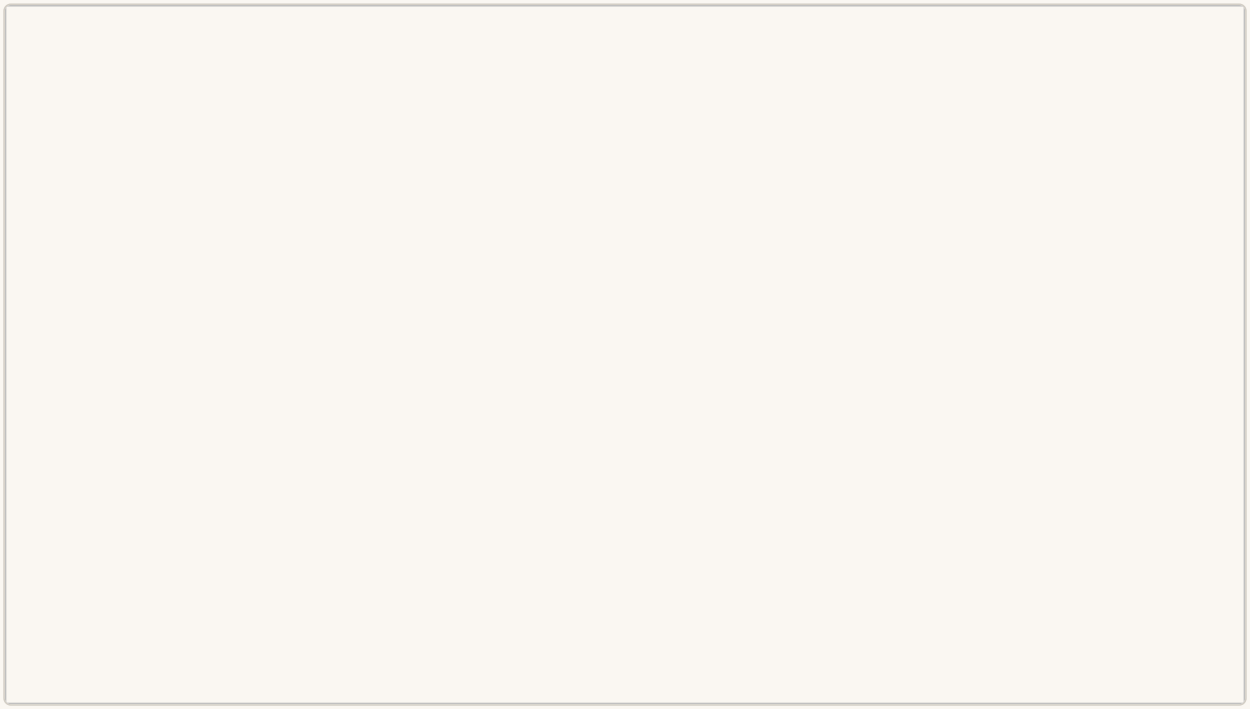
Files only. Then `powerbi-desktop reload` to show it.

The demo runs one prompt — "The layout of the Sales Overview page is a mess — fix it." — against the Old Kettle Sales report with both skills present. The house-style skill's fast path applies the theme, a 1920 × 1080 canvas, the brand rail with the bottle, a grid, its chart rules, KPI deltas and an answer-as-title, then validates the files. Power BI Desktop is only reloaded at the end to show the result.

Same report. Same data. Ours.

AFTER

Every page. Not just the ones you reviewed.



Before and after the house-style run, on the same report and the same data (Power BI Desktop screenshots, 8 October 2026). The agent applied the team's theme, page grid, brand rail, chart rules and business definitions — none of which it could have known without the functional skill.

Skills say what good looks like. Tools know how to get there.

SKILL

The intangible

Style · Meaning · Never do

+

TOOLS

The mechanics

CLI · MCP · Bridge

Skills and tools split the work. A skill holds what only your team knows — style, meaning, rules. Tools — command-line interfaces, MCP servers, the Power BI Desktop Bridge — already carry how to work with the files and the running application, so a skill does not need to repeat it. The next chapter is about those tools.

Choose the surface by what it exposes

TYPED SERVICE

MCP

`Power BI Authoring · Fabric`

Semantic model · remote data

PROJECT CLI

pbir-cli

`Power BI report files`

Repeatable project operations

RUNNING APPLICATION

Desktop Bridge

`powerbi-desktop`

Local Desktop state · reload · screenshot

Three distinct surfaces; choose the one that can reach the state you need.

MCP exposes structured services such as semantic-model operations and remote Fabric lookups. pbir-cli exposes repeatable Power BI report-project operations. The Desktop Bridge reaches the running Power BI Desktop application locally and is also available as `powerbi-desktop`. These surfaces can work together; they are not interchangeable.

MCP: operate on the semantic model

Natural-language intent can become model operations.

```
Intent → MCP + TOM → Model
```

Local server	VS Code extension
Bulk changes	Make repeatable edits and apply team practices
Check the target	Confirm the connection before an operation

The Power BI Authoring MCP Server is distributed as a local VS Code extension and exposes operations for semantic models. It enables repeatable changes such as organizing measures. Confirm the target connection and inspect the result after every operation.

CLI: make report-project work repeatable

PROJECT WORKFLOW

pbir-cli

```
01  inspect project
02  make a focused change
03  validate the result
```

Works with project files

Keep report changes alongside the PBIP project.

Repeatable

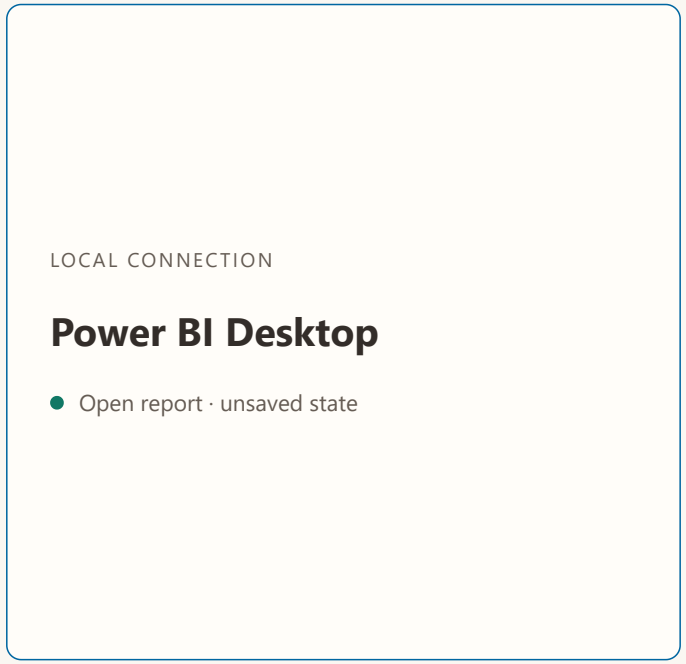
Use the CLI for explicit, scriptable operations.

Reviewable

Inspect the changed files before checking the page in Desktop.

pbir-cli is the project-file surface for repeatable report operations. Inspect the change and validate the project files; use Power BI Desktop to judge the rendered report.

Bridge: reach the running Desktop app



Inspect

Check the current application and report state.

Reload

Reload the report after its project files change.

Screenshot

Capture the page so a person can review the result.

`powerbi-desktop · status · reload · screenshot`

The Power BI Desktop Bridge is a local connection to the running application. It can inspect state, reload the report and capture screenshots through the `powerbi-desktop` CLI.

Power BI Desktop Bridge overview



DEMO LIVE plan prepared · confirm target before run

Bulk model cleanup

Check counts, expressions, references and existing visuals after the move.

The live, prepared model-cleanup plan uses the Power BI Authoring MCP Server to move measures into `_measures`, organize display folders and hide only corresponding raw columns. Verify the model and existing report visuals afterward.

Tools can check. You still decide what ships.

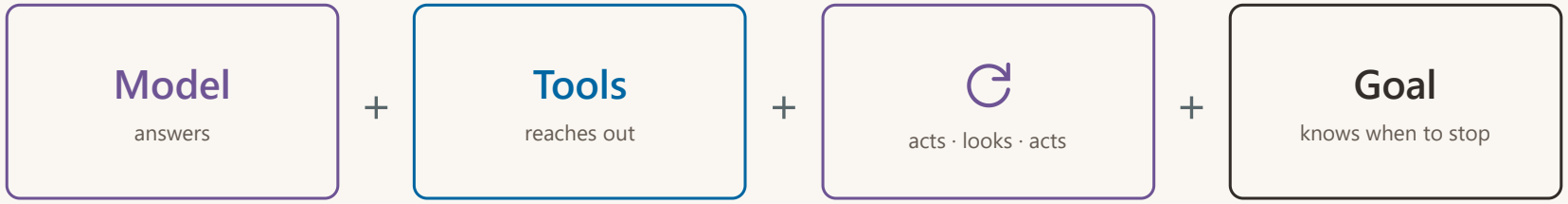
The agent can check what it made.

You still judge whether it is right.

"I made it good at my job. I'm still doing my job."

Tools let an agent inspect state and check the result of its work, but they do not remove the human decision about what is correct or ready to ship. This chapter hands off to Ch04 on the line: "I made it good at my job. I'm still doing my job."

What's an agent?



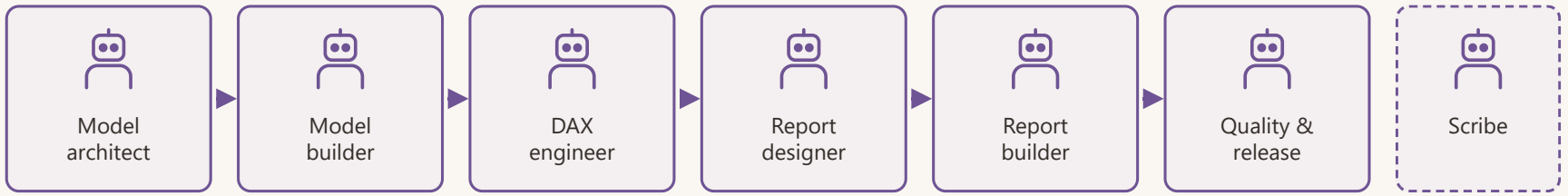
You just watched one: **Nicky's, in Desktop.**

An agent is a model in a loop with tools and a goal. A chat assistant answers; an agent acts, looks at the result, decides its next step and acts again until the goal is met. The previous chapter's Desktop work — edit, reload, look, fix — was an agent at work.

You're the head chef. This is your brigade.

Skills = the standard

Tools = the equipment



never approves its own work

THE PASS



"Release 2: where are we losing beer?"

A squad is a team of agents with one role each — here a Power BI team: model architect, model builder, DAX engineer, report designer, report builder, quality and release, plus a scribe that records decisions. Work moves station to station with evidence at each hand-off; the quality station never approves its own work. The human sets the standard (skills), provides the equipment (tools), places the order and approves the result.

bradygaster/squad

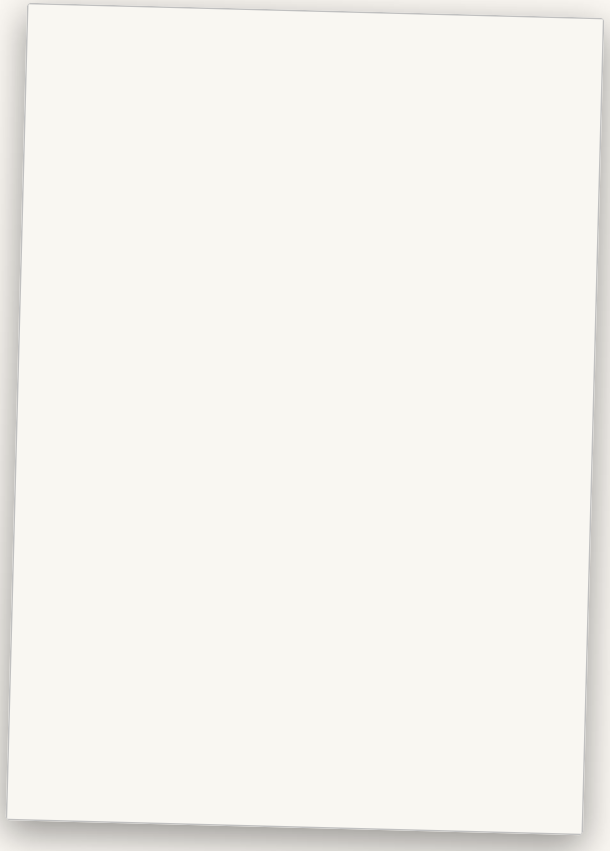
The order

THE HEAD BREWER · 8 OCTOBER

> "We lose beer between the kettle and the bottle. Which brewery is slipping compared with last year — and where do I start this month?"

Live: the sales model and "Revenue this month"

Warehouse: `Fact_Production` — 19,346 brew batches



The demo extends a report already in production. The request: find where beer is lost between brewing and packaging, by brewery, against last year. The inputs are the existing model and report, the production table in the warehouse (one row per brew batch) and a rough sketch of the model change and the new page.

DEMO LIVE backup: the dry run

The brigade at work

Power BI Squad | Autopilot | Claude Sonnet 5.5 · medium

We prepared the plan, so we get through it. The brigade picks it up.

7	Add `Production` to the model — reuse Date, Product, Location	model builder
8	Yield and quality, this month vs last year — and the title	DAX engineer
9	The Brewhouse page, in our house style	report builder · in parallel
10	Release: deploy, test every measure, reconcile with the warehouse	model builder

Then the diff — and `.squad/decisions.md`, the team's memory in git.

The squad — selected as the "Power BI Squad" agent in GitHub Copilot, on Autopilot so it does not pause for approval on each step — runs a prepared plan for release 2: the model builder adds the production table and reuses the existing date, product and location tables; the DAX engineer writes yield and quality measures for this month against last year, while the report builder adds the Brewhouse page in the house style in parallel; the model builder then releases, tests every measure and reconciles the results with the warehouse. The change set is reviewable as a normal diff, and accepted decisions are kept in the repository

Why a squad?

Speed

Work runs in parallel — the page is built while the measures are written.

Specialists

One task per agent, done properly — and each one tuned for its task.

Process

Separate steps, separate owners — a process you can see and improve.

Three reasons to use several specialised agents rather than one: speed, because independent steps run in parallel; quality, because each agent has one task with only the tools and instructions it needs and can be tuned for it; and process, because separate steps with separate owners make the work observable and improvable one step at a time.

Monday

START VS Code + Copilot Free, on a sample PBIP.

ADD `skills-for-fabric`

WRITE One skill — how *your* team works.

"It's a very fast brigade with no taste. That's what you're for."

To start: Visual Studio Code with GitHub Copilot Free (agent mode and MCP are included) on a sample Power BI project; add Microsoft's skills-for-fabric for the technical side; then write one functional skill describing how your team works. If your organisation manages Copilot centrally, talk to IT rather than working around it, and keep company data out of personal accounts.

GitHub Copilot plans / microsoft/skills-for-fabric